

ESERCITAZIONE 1. - A.A. 2016-2017

CALCOLO E RAPPRESENTAZIONE DI ORBITE TRIDIMENSIONALI

VERSIONE 1.6 - 12/10/2016

1. TESTO DELL'ESERCITAZIONE

Utilizzando il linguaggio di programmazione MATLAB, lo studente codifichi un algoritmo atto a risolvere i seguenti problemi:

Parte 1: Dati posizione e velocità iniziali di un satellite in un riferimento inerziale ed un vettore di epoche, calcolare posizione e velocità del satellite in corrispondenza di ciascuna epoca e rappresentare graficamente la traiettoria del satellite nel riferimento inerziale, salvando una copia su file.

Parte 2: Assumendo che la Terra sia sferica e ruoti con velocità angolare costante, calcolare e rappresentare graficamente la traccia a terra di un satellite di cui è nota la posizione nel riferimento inerziale in un certo numero di istanti temporali.

Assumendo a titolo di esempio che un satellite si trovi il 15/01/1995 @ 00:00:00 nelle condizioni di Tabella 1, si applichino i programmi precedentemente sviluppati per calcolare traiettoria orbitale e traccia a terra nelle successive 24 ore.

GPS	r_0 [km]	13426.726354	-20645.023752	10108.071628
	v_0 [km/s]	2.638622	0.329454	-2.805018
Molniya	r_0 [km]	2869.944136	-1656.963019	-6617.757380
	v_0 [km/s]	4.814217	8.338469	0.000000
Geosincrona	r_0 [km]	-29260.792529	23865.119693	23865.119693
	v_0 [km/s]	-2.334835	-1.212853	-1.212853

TABELLA 1. Condizioni Iniziali

Parte 3: Prendendo come riferimento la missione 2 (orbita Molniya), valutare analiticamente e verificare numericamente, quanto tempo spende il satellite ad una latitudine superiore a 45° .

Parte 4: Progettare una costellazione di tre satelliti Molniya, di periodo $12 \text{ h}^{\text{sid}}$, in modo che i satelliti abbiano la stessa traccia a terra, siano distribuiti uniformemente, e siano tali per cui al tempo t_A corrispondente al 15/1/1995 @ 00:00:00, un satellite sia all'apogeo della propria orbita e ad una longitudine pari a quella di Mosca ($55^\circ 45' 06''\text{N}$; $37^\circ 37' 04''\text{E}$). Si rappresentino le orbite dei tre satelliti in un riferimento inerziale e la traccia a terra della costellazione nell'intervallo di tempo $t \in [t_A - 4h, t_A + 4h]$.

Dati:

- $\mu = 398600.4415 \text{ km}^3/\text{s}^2$
- $r_\oplus = 6378.1363 \text{ km}$
- $\alpha_{G0} = 100^\circ$ il 1/1/1995 @ 00:00:00
- $T_{\text{sid}} = 86164 \text{ s}$

2. SCHEMA DI RISOLUZIONE

Parte 1.

Creare le funzioni “base” per la soluzione di problemi di meccanica orbitale: **rvECI2coe**, **rvECI2coe**, **anu2AE**, **AE2XM**, **XM2AE**, **AE2anu**. usando le interfacce suggerite.

```
function COE = rvECI2coe(rVect, vVect, amu)
function [rVect, vVect] = rvECI2coe(COE, amu)
function AE = anu2AE(ecc, anu)
function XM = AE2XM(ecc, AE)
function AE = XM2AE(ecc, XM)
function anu = AE2anu(ecc, AE)
```

Creare una funzione CorePropagator

```
function [r1, v1] = CorePropagator( r0, v0, dt)
```

che assegnata una coppia qualunque r_0, v_0 riferita ad un'epoca t_0 , calcoli posizione r_1 e velocità v_1 del satellite dopo un tempo dt .

Creare una funzione ShellPropagator

```
function [rMatrixECI, vMatrixECI] = ShellPropagator( r0, v0, tspan)
```

che, assegnato un vettore di epoche $tspan = [t_0, t_1, t_2, \dots, t_{n-1}]$, valuti la posizione/velocità del satellite agli istanti corrispondenti $\{ (r_0, v_0), (r_1, v_1), \dots, (r_{n-1}, v_{n-1}) \}$ usando l'algoritmo sviluppato in precedenza e li salvi nelle matrici $rMatrixECI, vMatrixECI$ di dimensioni $3 \times n$.

Creare una funzione DisegnaTraj3D

```
function DisegnaTraj3D(rMatrixECI)
```

che mostri a schermo una figura con la traiettoria del satellite in un riferimento inerziale e salvarla su file.

Di seguito sono raccolti gli pseudocodici delle funzioni più complesse:

Algorithm 1 Pseudocode for CorePropagator

```
function COREPROPAGATOR( $r_0, v_0, dt$ )
     $coe \leftarrow rv2coe(r_0, v_0)$  ▷ ricavo elementi orbitali classici
     $AE_0 \leftarrow anu2AE(e, anu_0)$  ▷ calcolo anomalia eccentrica al tempo  $t_0$ 
     $XM_0 \leftarrow AE2XM(e, AE_0)$  ▷ calcolo anomalia media al tempo  $t_0$ 
     $XM_1 \leftarrow \{\dots\}$  ▷ calcola anomalia media al tempo  $t_1$ 
     $AE_1 \leftarrow XM2AE(e, XM_1)$  ▷ calcolo anomalia eccentrica al tempo  $t_1$ 
     $anu_1 \leftarrow AE2anu(e, AE_1)$  ▷ calcolo anomalia vera al tempo  $t_1$ 
     $coe \leftarrow \text{update } coe \text{ at } t_1$  ▷ aggiorni gli elementi orbitali (solo l'anomalia vera!)
     $r, v \leftarrow coe2rvECI(coe)$  ▷ ricavo pos/vel dagli elementi orbitali aggiornati
end function
```

Algorithm 2 Pseudocode for ShellPropagator

```
function SHELLPROPAGATOR( $r_0, v_0, tspan$ )
    for  $i \leq n$  do
         $dt \leftarrow tspan(i) - tspan(1)$ 
         $r, v \leftarrow \text{CorePropagator}(r_0, v_0, dt)$  ▷ propago da  $t_0$  a  $tspan(i)$ 
         $rMatrixECI(:, i) \leftarrow r;$ 
         $vMatrixECI(:, i) \leftarrow v;$ 
    end for
end function
```

Algorithm 3 Pseudocode for DisegnaTraj3D

```

function DISEGNATRAJ3D(rMatrixECI)
    figure()                                ▷ nuova figura
    definisci il layout                    ▷ (viewpoint, x,y,z-label, title)
     $X, Y, Z \leftarrow \text{rMatrixECI}$ 
    plot3(X,Y,Z,'-b')                      ▷ disegna l'orbita
    save pdf
end function

```

Parte 2.

Creare una funzione **ECI2ECEF**

```
function [rVectECEF] = ECI2ECEF(JD, rVectECI)
```

che trasforma un vettore posizione in componenti ECI nelle corrispondenti componenti valutate in un riferimento Earth-Centered Earth-Fixed, all'epoca JD.

Creare una funzione **rect2polar**

```
function [r, th, fi] = rect2polar(rVect)
```

che trasforma un set di componenti cartesiane in componenti polari (modulo, longitudine, latitudine).

Creare una funzione **CoreTrace**

```
function [Lat, Long] = CoreTrace(JD, rVectECI)
```

che, dato un singolo vettore di posizione nel riferimento inerziale rECI riferito all'epoca JD, ne calcoli l'impronta a Terra.

Creare una funzione **ShellTrace**

```
function [LatVect, LongVect] = ShellTrace(rMatrixECI, tspan_MJD)
```

che, assegnato un vettore $1 \times n$ di epoche $t_{\text{span_MJD}} = [t_1, t_2, \dots, t_{n-1}]$, ed una matrice $3 \times n$ rMatrixECI contenente la posizione del satellite nel riferimento inerziale negli istanti corrispondenti, valuti per ciascuno longitudine e latitudine L_o, L_a usando l'algoritmo sviluppato in precedenza e li salvi nelle matrici LongVect, LatVect di dimensioni $1 \times n$.

Creare una funzione DisegnaTraccia2D

```
function DisegnaTraccia2D(LatVect, LongVect)
```

che mostri a schermo (e salvi su file) una figura con la traccia a terra del satellite nel piano Longitudine-Latitudine.

Di seguito sono raccolti gli pseudocodici delle funzioni più complesse:

Algorithm 4 Pseudocode for CoreTrace

```

function CORETRACE(MJD, rVectECI)
    rVectECEF  $\leftarrow$  ECI2ECEF(MJD, rVectECI)
    Lat, Long  $\leftarrow$  rect2polar(rVectECEF)
end function

```

Algorithm 5 Pseudocode for ShellTrace

```

function SHELLTRACCIA(rMatrixECI, tspan)
    for  $i \leq n$  do
        rVectECI  $\leftarrow$  rMatrixECI(:,i)
        Lat, Lon  $\leftarrow$  CoreTrace(MJD, rVectECI)
        LatVect(i)  $\leftarrow$  Lat;
        LongVect(i)  $\leftarrow$  Lon;
    end for
end function

```

Algorithm 6 Pseudocode for ECI2ECEF

function ECI2ECEF(MJD,rVectECI)

Define problem constant

 $\triangleright$ solar day, sidereal day, Earth rotation rate Evaluate α_{grw} $\triangleright$ Right ascension of Greenwich Meridian $T_{ECI2ECEF} \leftarrow R_3(+\alpha_{grw})$ $\triangleright$ Rotation matrix from ECI to ECEF. $\triangleright$ NB: $+\alpha_{grw}$ rVectECEF $\leftarrow T_{ECI2ECEF}$ rVectECI**end function**

Nomi delle variabili usate nel codice:

rVectECI: $[3 \times 1]$ S/C position in ECIvVectECI: $[3 \times 1]$ S/C velocity in ECIrVectECI: $[3 \times 1]$ S/C position in ECEF

amu: grav. parameter of the primary body

COE: classical orbital elements

[a; ecc; ainc; gom; pom; anu]

ainc: inclination

gom: right ascension of the ascending node (RAAN)

pom: perigee argument

anu: true anomaly

AE: eccentric anomaly

XM: mean anomaly

MJD: Modified Julian Date

tspan_MJD: $[1 \times n]$ vector of epochs in seconds since 1/1/2000 @ 00:00:00tspan: $[1 \times n]$ vector of epochs in MJD

Consigli:

- (1) Usare sempre vettori colonna per le variabili posizione e velocità;
- (2) Per esportare figure in formato pdf usare il comando:

```
iResolution = 600;
ImgNameStr = 'pic1';
print('-dpdf', sprintf('-r%d', iResolution), strcat(ImgNameStr, '.pdf'));
```

- (3) **NEW**

Per esportare figure si può usare la funzione saveTightFigure di E Akbas, usando la sintassi:

```
hFig = figure(n) %figure handle
saveTightFigure(hFig, outputFileName) %Non usare gcf come figure handle!
```